

AI-Driven SOC Alert Triage Automation

Integrating Wazuh with n8n AI Agents for Intelligent Security Operations

Project Overview

This project demonstrates the practical implementation of **AI-powered SOC automation** by integrating **Wazuh SIEM** with **n8n workflow automation** and an **AI Agent (Google Gemini)** to automate alert triage.

The goal was to reduce SOC analyst workload by automatically:

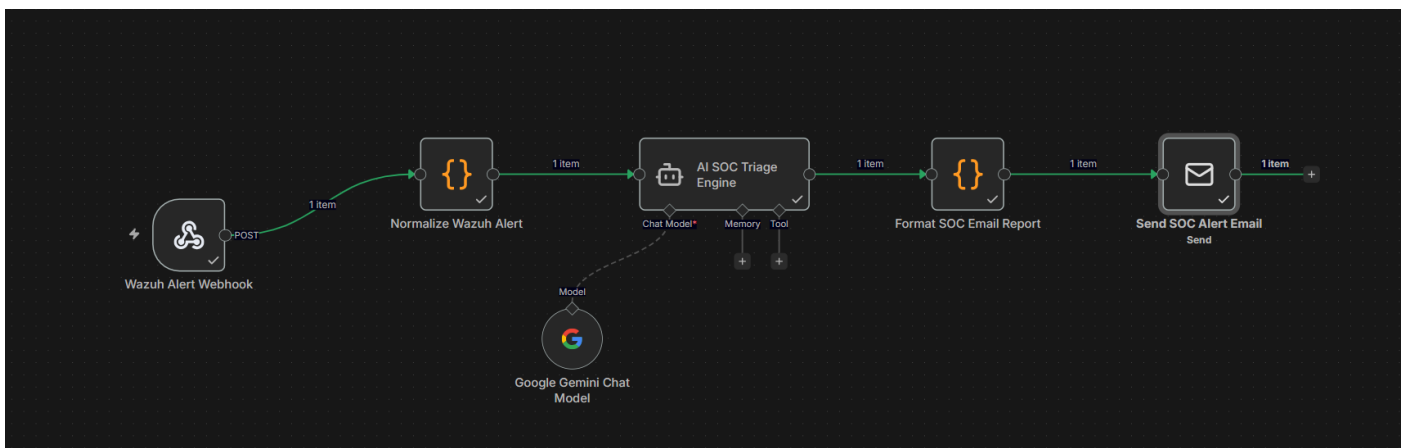
- Receiving real-time alerts from Wazuh
- Filtering high-severity alerts
- Sending alerts to an AI agent
- Generating structured triage analysis
- Producing visually formatted SOC email reports
- Delivering actionable intelligence to analysts

Author: Abdul Rehman

AI Automation | Cybersecurity | SOC | AWS
Security Projects

[LinkedIn](#)

This project simulates a **real-world production SOC automation pipeline**.



wazuh.

+

n8n

Architecture Overview

Wazuh Manager

- Custom Integration Script
- n8n Webhook Trigger
- Alert Normalization
- AI SOC Triage Engine (Gemini Model)
- Email Report Formatter
- SMTP Email Delivery to Analyst

Tools & Technologies Used

- **Wazuh SIEM**
- **n8n Automation Platform**
- **Google Gemini AI Model**
- **SMTP (Gmail App Password)**
- **Linux (Wazuh Manager Configuration)**
- **JavaScript (Data Normalization & Email Formatting)**

Why Integrate Wazuh with n8n AI Automation?

❖ **Faster Triage**

AI automatically summarizes alerts, reducing manual log review.

❖ **Reduced Alert Fatigue**

Only high-severity alerts are forwarded and intelligently analyzed.

❖ **Automated SOC Workflow**

End-to-end automation from detection to analyst notification.

❖ **Reduced MTTD & MTTR**

- Faster Mean Time to Detect
- Faster Mean Time to Respond

❖ **Actionable Intelligence**

AI provides:

- Summary
- Risk Assessment
- Recommended Actions
- Clear Next Step

❖ **SOAR Learning Experience**

Hands-on implementation of:

- Security Orchestration

- Automation
- AI-driven Response

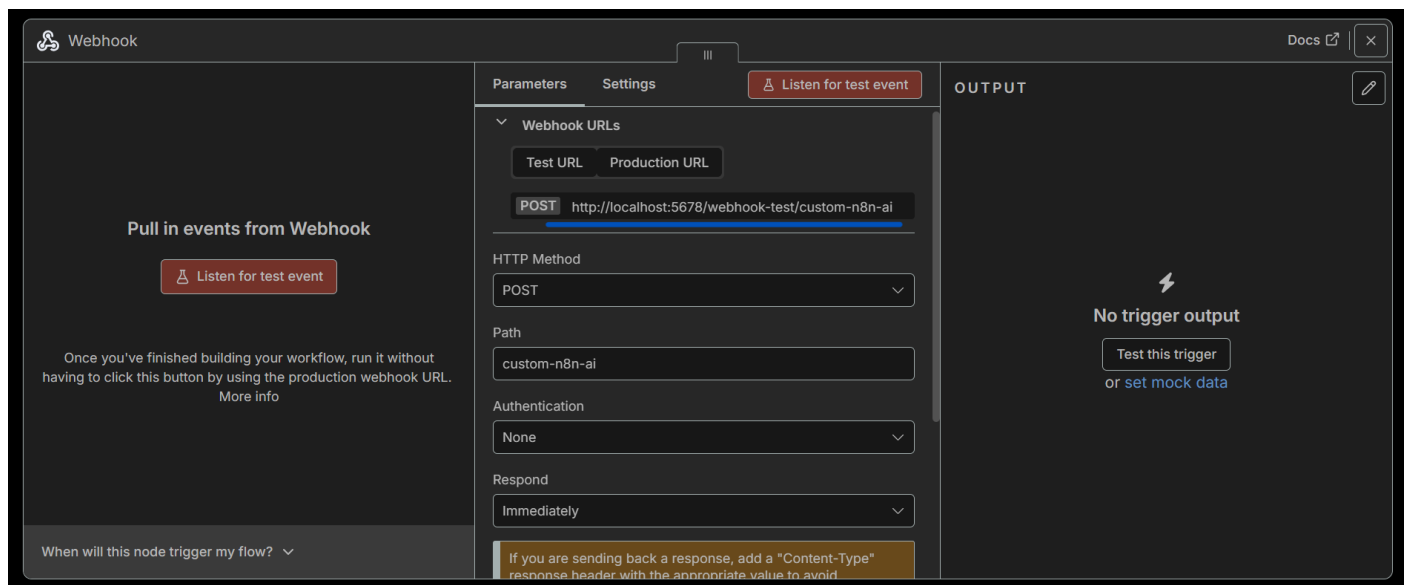
Step-by-Step Implementation

Step 1: Configure n8n Webhook Trigger

- Created a Webhook node in n8n.
- Copied the production webhook URL.
- This URL will receive alerts from Wazuh.

Purpose:

To allow Wazuh to push alerts into n8n workflow.



Step 2: Create Custom Wazuh Integration Script

- Developed a custom integration script inside: `/var/ossec/integrations/`
- Script filters alerts:
 - Only severity level ≥ 7 are forwarded.

Purpose:

Ensure only meaningful alerts are sent for AI triage.

```
GNU nano 7.2 custom-n8n-ai *
ONLY alerts with rule.level >= 7 are forwarded
"""

# =====
# HIGH SEVERITY FILTER (IMPORTANT)
# =====
level = alert.get('rule', {}).get('level', 0)

if level < 7:
    return ""

# Severity mapping (kept as-is)
if level <= 4:
    severity = "1 low"
elif 5 <= level <= 7:
    severity = "2 normal"
else:
    severity = "3 high"

full_log = alert.get('full_log', 'N/A').replace("\n", " ")

msg = {
```

Step 3: Set Script Permissions

Used:

```
root@wazuhserver:/var/ossec/integrations# sudo chown root:wazuh /var/ossec/integrations/custom-n8n-ai
root@wazuhserver:/var/ossec/integrations# sudo chmod 750 /var/ossec/integrations/custom-n8n-ai
```

Purpose:

Allow Wazuh Manager to execute the integration script securely.

Step 4: Add Custom Integration Block in ossec.conf

Added:

```
<integration>
  <name>custom-n8n-ai</name>
  <hook_url>http://localhost:5678/webhook-test/custom-n8n-ai</hook_url>
  <level>7</level>
  <alert_format>json</alert_format>
</integration>
```

Purpose:

Link Wazuh alert engine to n8n webhook.

Step 5: Verify Alert Forwarding

- Monitored integration logs.
- Confirmed alerts successfully sent to n8n.

Purpose:

Validate connectivity and successful integration.

```
/tmp/custom-n8n-ai-1770779776--1015093122.alert http://localhost:5678/webhook-test/custom-n8n-ai
/tmp/custom-n8n-ai-1770779777--2068502060.alert http://localhost:5678/webhook-test/custom-n8n-ai
/tmp/custom-n8n-ai-1770779778--1698645930.alert http://localhost:5678/webhook-test/custom-n8n-ai
/tmp/custom-n8n-ai-1770779779--561808702.alert http://localhost:5678/webhook-test/custom-n8n-ai
```

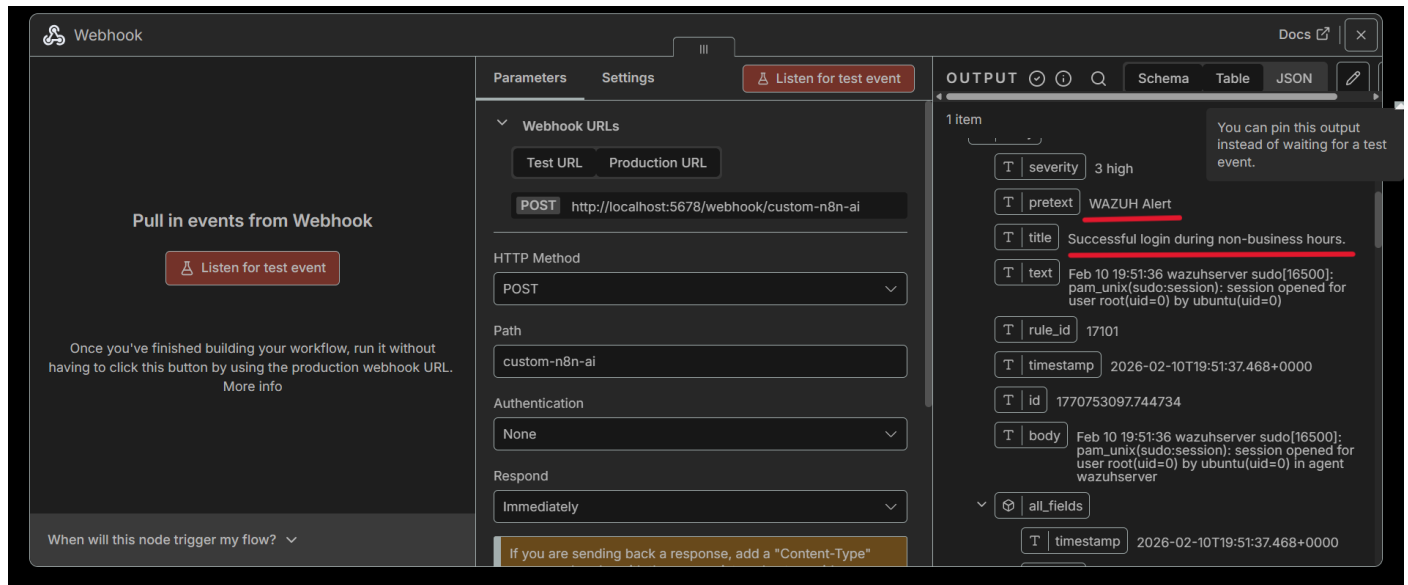
Step 6: Validate Webhook Data in n8n

Webhook successfully received:

- Full alert JSON
- Rule metadata
- MITRE mappings
- Agent details

Purpose:

Confirm alert schema structure is correct.



The screenshot shows the n8n Webhook node configuration and its output. The configuration is set to listen for POST requests at the URL `http://localhost:5678/webhook/custom-n8n-ai`. The output shows a single item with the following fields:

Field	Value
severity	3 high
pretext	WAZUH Alert
title	Successful login during non-business hours.
text	Feb 10 19:51:36 wazuhsrver sudo[16500]: pam_unix(sudo:session): session opened for user root(uid=0) by ubuntu(uid=0)
rule_id	17101
timestamp	2026-02-10T19:51:37.468+0000
id	1770753097.744734
body	Feb 10 19:51:36 wazuhsrver sudo[16500]: pam_unix(sudo:session): session opened for user root(uid=0) by ubuntu(uid=0) in agent wazuhsrver

Step 7: (Initial) IF Node Filtering

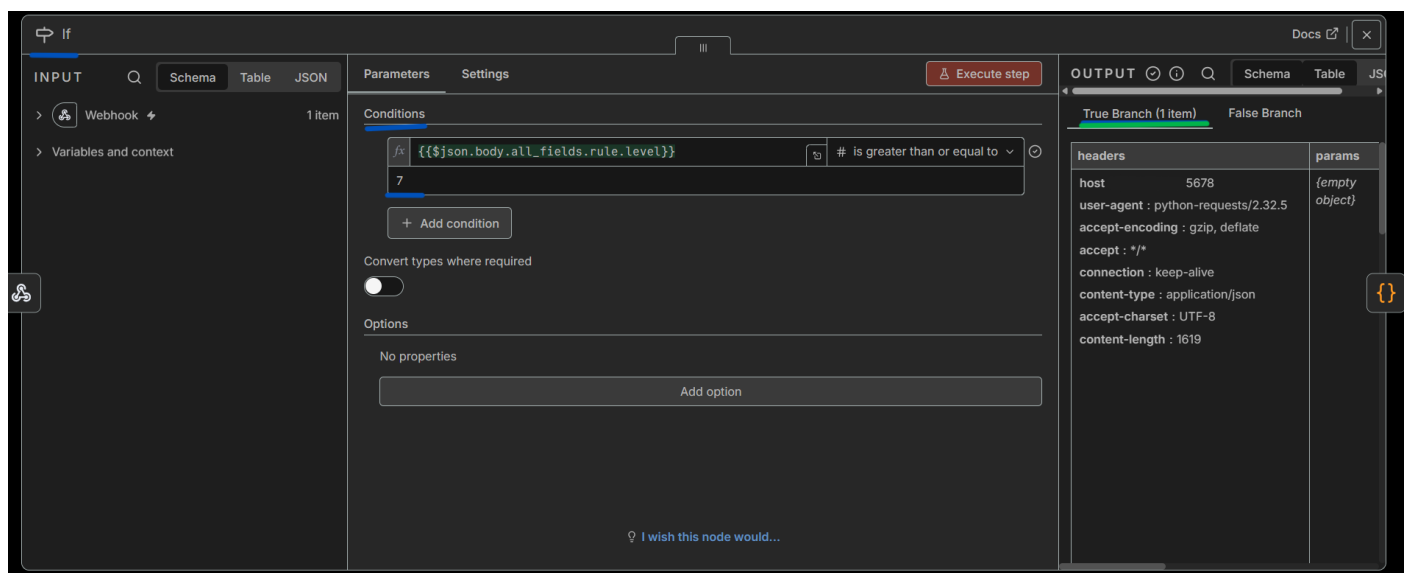
Added IF condition:

- Forward only `rule.level ≥ 7`

Later removed because filtering already done in script.

Lesson Learned:

Avoid redundant filtering when already enforced upstream.



The screenshot shows the n8n IF node configuration. The condition is set to `{{ $json.body.all_fields.rule.level }} # is greater than or equal to 7`. The output shows the True Branch (1 item) with the following headers and parameters:

headers	params
host: 5678	{empty object}
user-agent: python-requests/2.32.5	
accept-encoding: gzip, deflate	
accept: */*	
connection: keep-alive	
content-type: application/json	
accept-charset: UTF-8	
content-length: 1619	

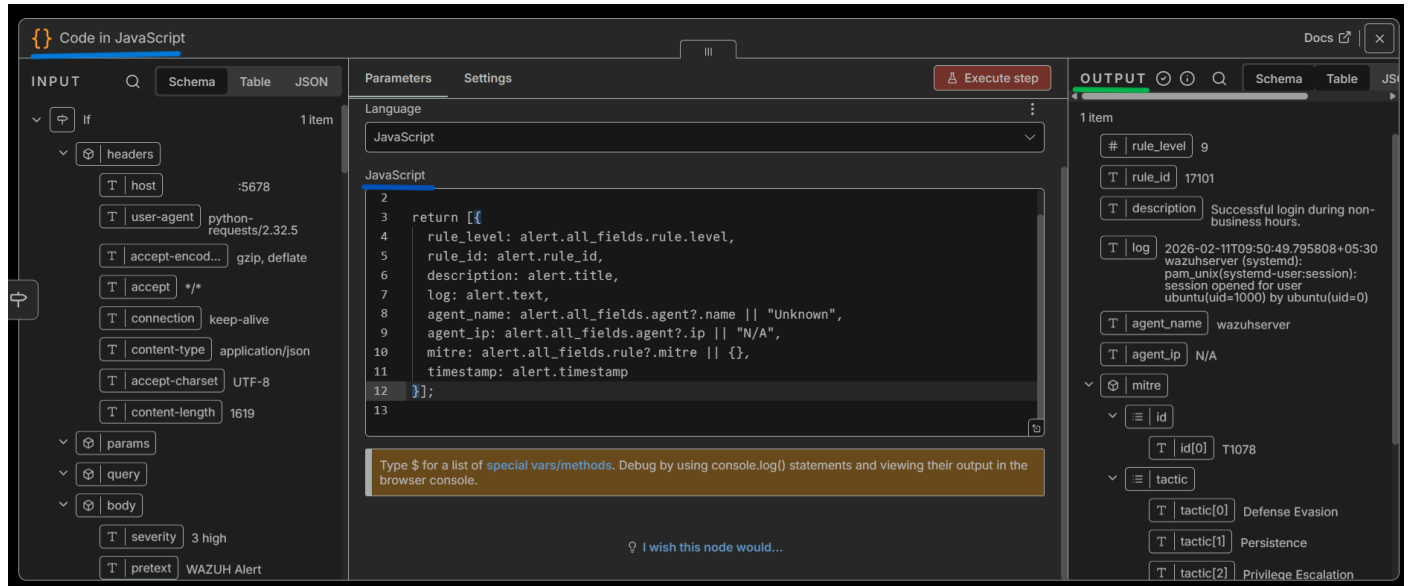
Step 8: Normalize Alert Using JavaScript

Created a “Normalize Wazuh Alert” node:

- Extracted important fields
- Structured JSON for AI
- Later modified to pass full raw alert context

Purpose:

Provide structured and contextual data for AI analysis.



The screenshot shows a Node-RED workspace with a JavaScript node configured to process Wazuh alerts. The node is named "Code in JavaScript" and is set to "JavaScript" language. The input is a "WAZUH Alert" with a severity of "3 high". The JavaScript code defines a function that returns a structured JSON object containing alert details. The output of the node is a single item with the following structure:

```

{
  "#": "rule_level",
  "rule_level": 9,
  "rule_id": 17101,
  "description": "Successful login during non-business hours.",
  "log": "2026-02-11T09:50:49.795808+05:30\nwazuhserver (systemd):\npam_unix(systemd-user:session):\nsession opened for user\nubuntu(uid=1000) by ubuntu(uid=0)",
  "agent_name": "wazuhserver",
  "agent_ip": "N/A",
  "mitre": {
    "id": "T1078",
    "tactic": [
      "Defense Evasion",
      "Persistence",
      "Privilege Escalation"
    ]
  }
}

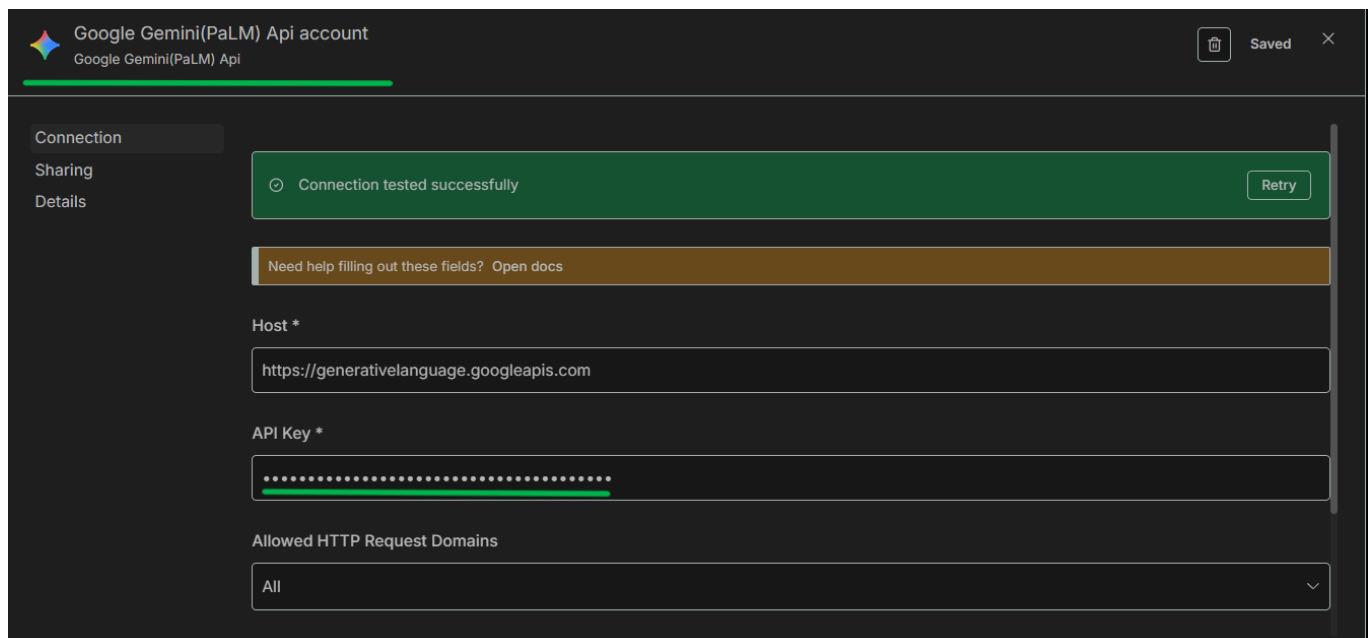
```

Step 9: Configure AI SOC Triage Engine

- Integrated Google Gemini model
- Added API credentials
- Connected to AI Agent node

Purpose:

Enable intelligent alert triage using LLM.



The screenshot shows the Google Gemini (PaLM) API account configuration page. The page is titled "Google Gemini(PaLM) Api account" and "Google Gemini(PaLM) Api". It has a "Saved" button in the top right corner. The left sidebar contains "Connection", "Sharing", and "Details" tabs. The main content area shows a "Connection" section with a green status bar indicating "Connection tested successfully" and a "Retry" button. Below this is a link to "Open docs" for help. The configuration fields include:

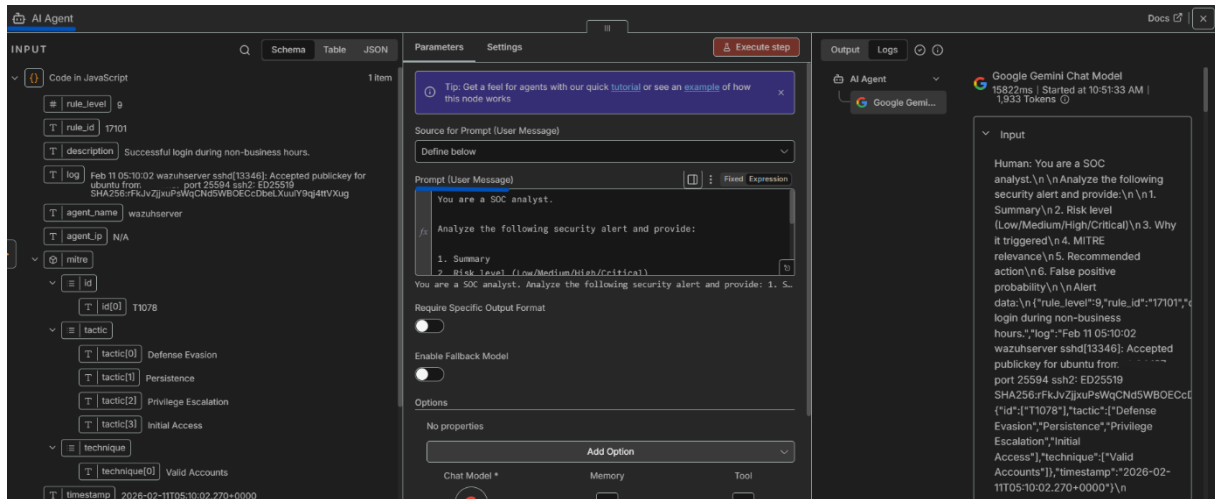
- Host ***: `https://generativelanguage.googleapis.com`
- API Key ***: A field with a green underline and a series of dots representing the API key.
- Allowed HTTP Request Domains**: A dropdown menu set to "All".

Step 10: Initial AI Prompt

- Basic prompt
- Limited instructions
- Resulted in messy and unstructured output

Problem Identified:

- Broken formatting
- Irrelevant content
- Inconsistent analysis



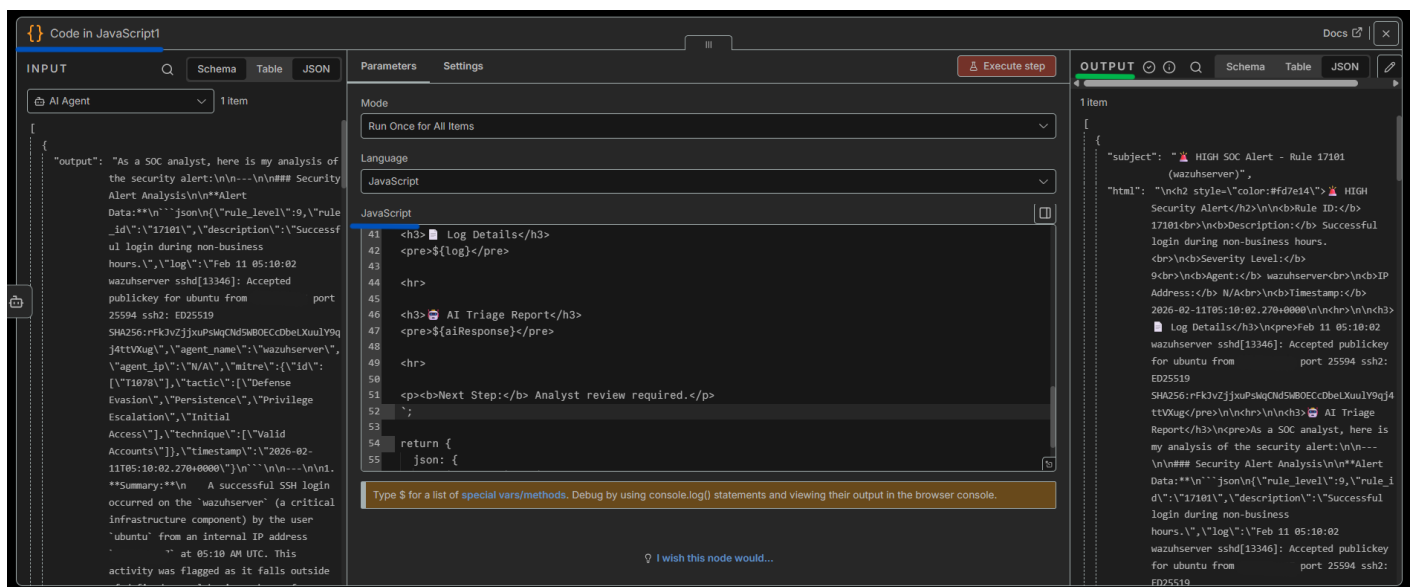
Step 11: Create Email JSON Formatter

Built JavaScript node to:

- Extract AI response sections
- Format structured HTML
- Design SOC-style report layout

Purpose:

Convert AI text into professional SOC email report.

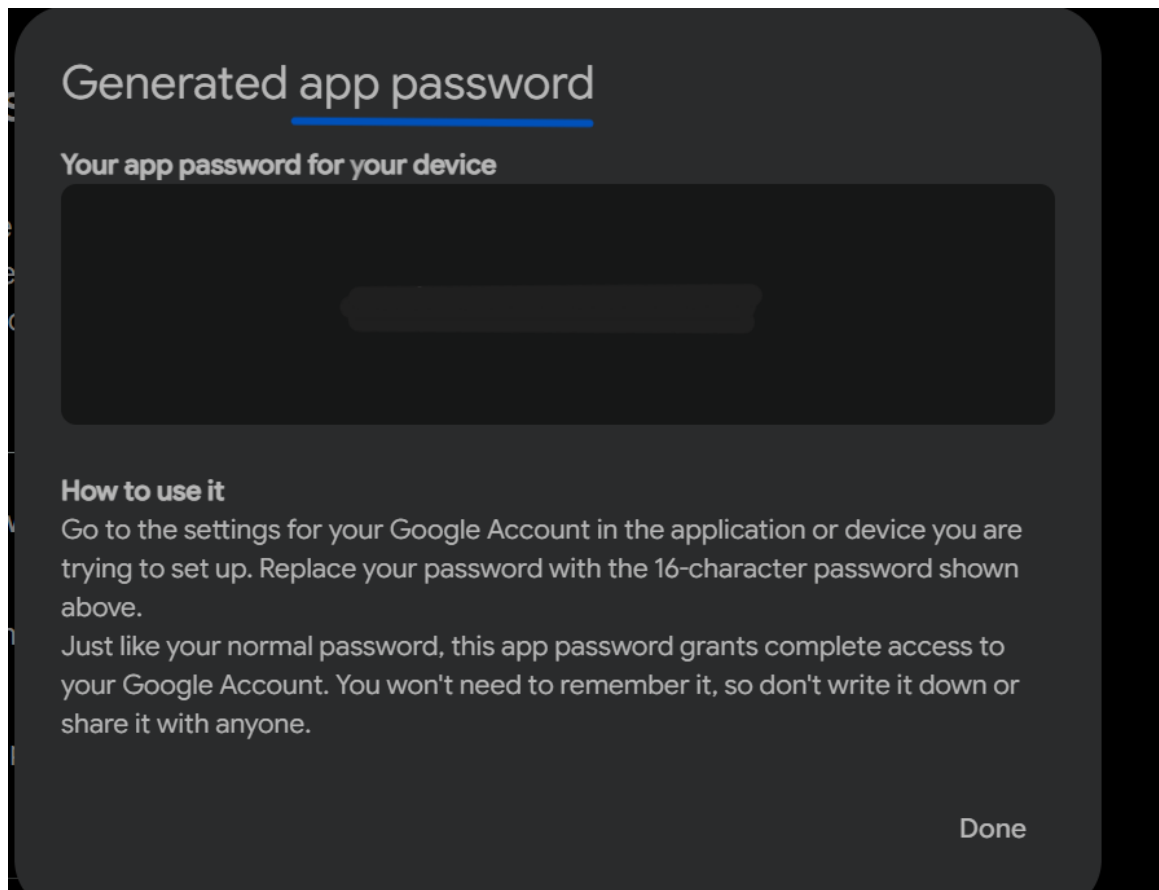
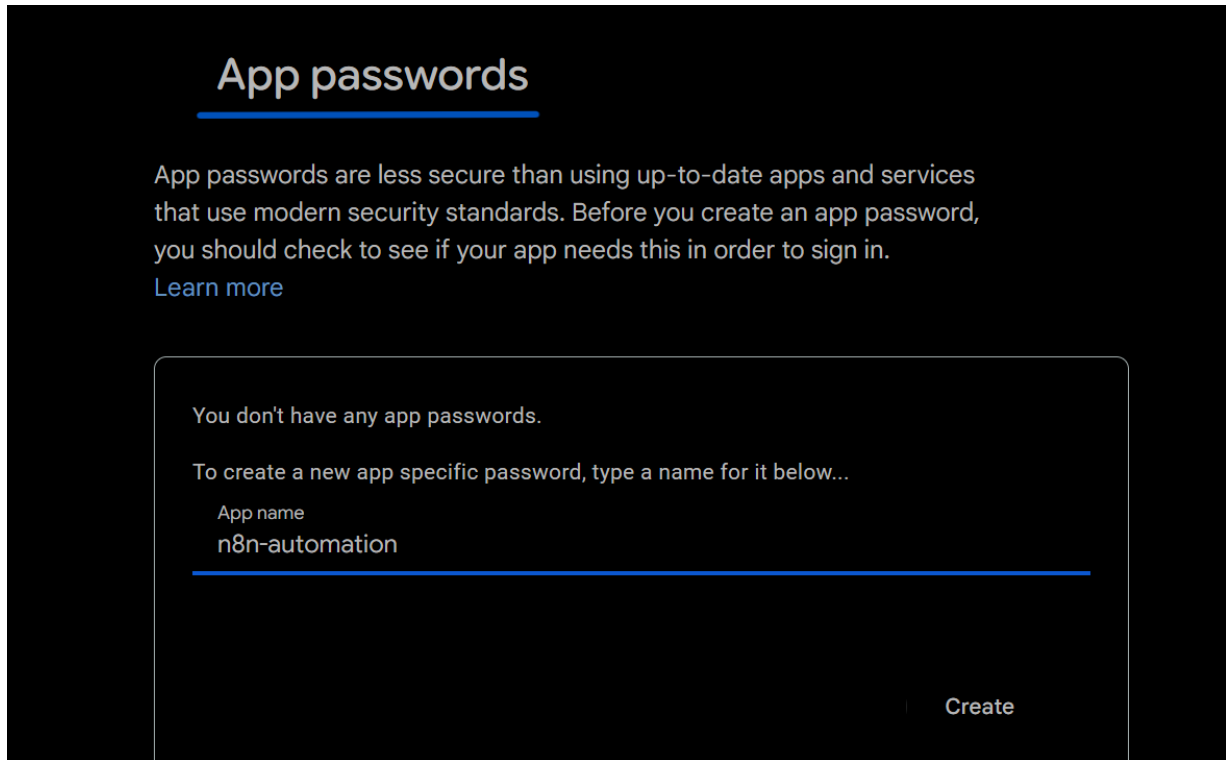


Step 12–13: Setup Gmail App Password

- Generated Google App Password
- Configured SMTP credentials in n8n

Purpose:

Secure email delivery without exposing main password.



Step 14: Configure Send Email Node

Mapped:

- Subject
- Severity level
- Alert overview
- AI triage report
- Log details

Purpose:

Send automated triage report to SOC analyst.

SMTP account
SMTP

Connection
Sharing
Details

Connection tested successfully
Retry

Need help filling out these fields? Open docs

User

@gmail.com

Password

.....

Host

smtp.gmail.com

Port

465

Send email

Schema
Table
JSON

Code in JavaScript

1 item

```

{
  "subject": " HIGH SOC Alert - Rule 17101 (wazuhserver)",
  "html": "
<h2 style='color:#d7e144'> HIGH Security Alert</h2>
<h3>Rule ID: 17101</h3>
<div>
<div>Severity Level:</div>
<div>9</div>
<div>Agent:</div>
<div>WazuhServer</div>
<div>IP Address:</div>
<div>11705:10:02:270:0000</div>
<div>Log Details:</div>
<div>Feb 11 05:10:02 wazuhserver sshd[13346]: Accepted publickey for ubuntu from  port 25594 ssh2: ED25519
SHA256:rfKvZj3uPmQcM5MOECc0BelXuiY9qJ4ttVXug</pre>

```

Parameters

Settings

Execute step

Credential to connect with

SMTP account

Operation

Send

From Email

@gmail.com

To Email

@gmail.com

Subject

{{ \$json.subject }}

HIGH SOC Alert - Rule 17101 (wazuhserver)

Email Format

HTML

HTML

{{ \$json.html }}

<h2 style='color:#d7e144'> HIGH Security Alert</h2>
<div>Rule ID:</div>
<div>17101</div>
<div>Desc-

Options

No properties

Add option

I wish this node would...

OUTPUT

1 item

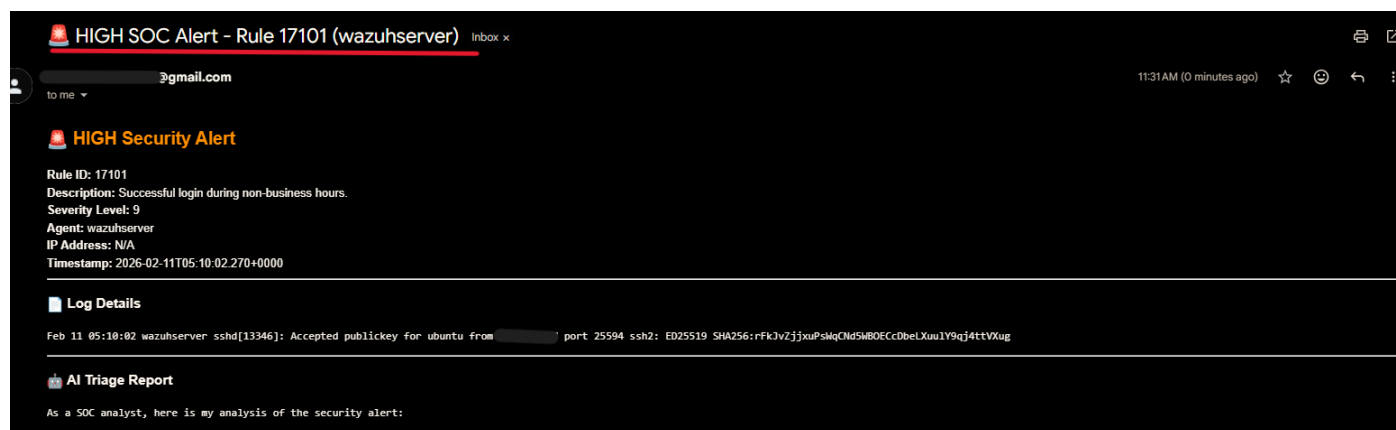
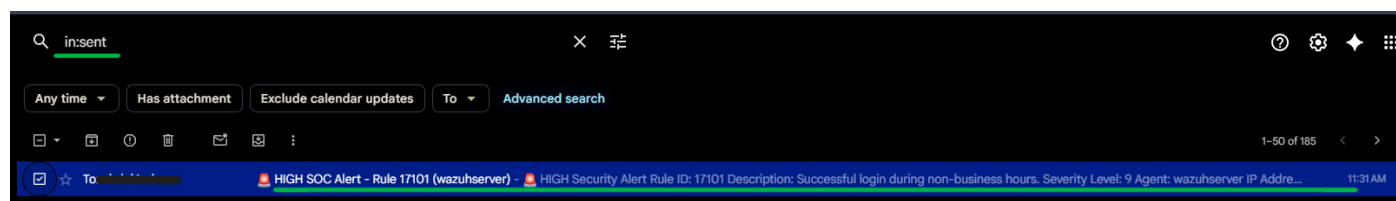
```

{
  "accepted": [
    {
      "email": "@gmail.com"
    }
  ],
  "rejected": [
  ],
  "ehlo": [
    "SIZE 35882577",
    "8BITMIME",
    "AUTH LOGIN F ",
    "ENHANCEDSTATUSCODES",
    "PIPELINING",
    "CHUNKING",
    "SMTPUTF8"
  ],
  "envelopeTime": 1163,
  "messageTime": 525,
  "messageSize": 8557,
  "response": "250 2.0.0 OK 1770789677 2adb3069b0e04-59e5f5ac749a118335e87.66 -",
  "smtp": {
    "envelope": {
      "from": "@gmail.com",
      "to": [
        "@gmail.com"
      ]
    },
    "messageId": "ce350f40a-f049-64b0-31d7-d8876b53e6c@gmail.com"
  }
}

```

Step 15–16: Email Delivery Validation

- Confirmed email sent from SMTP
- Verified inbox receipt by SOC analyst



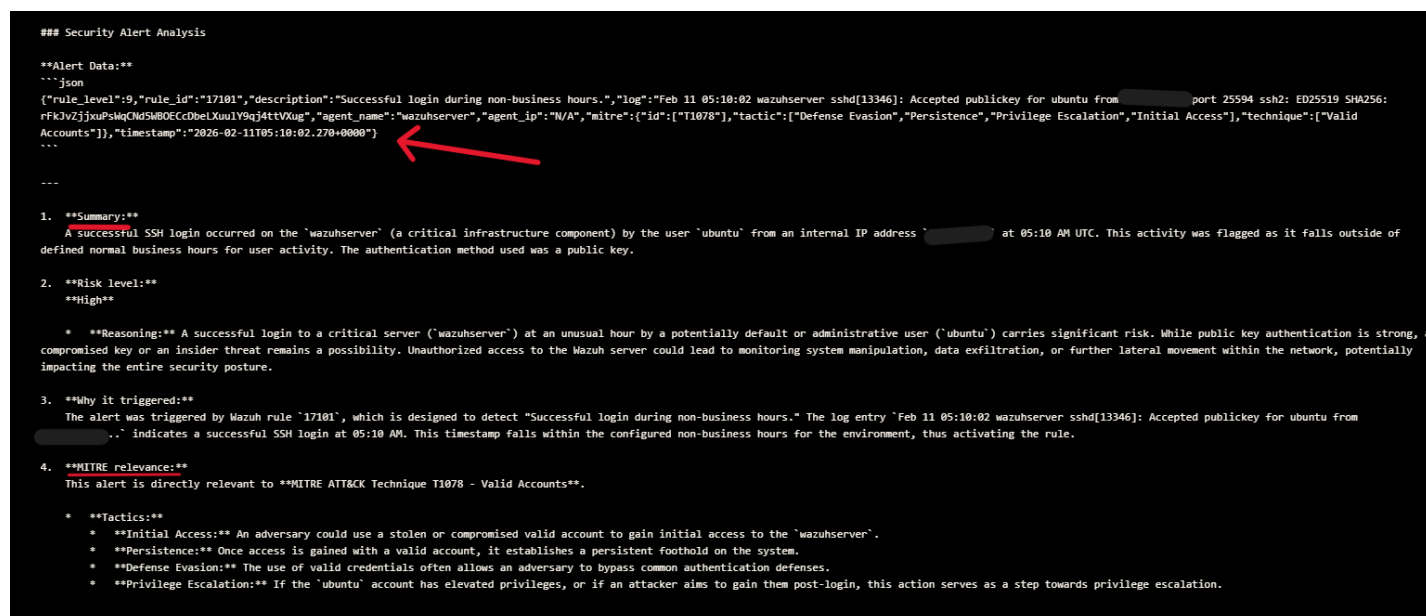
Step 17: Identified Formatting Issues

Observed:

- Broken AI formatting
- Unwanted quotes
- Overly long output
- Improper risk assessment section

Root Cause:

Weak AI prompt engineering.



```

1. **Immediate Verification (High Priority):**
  * **Identify User 'ubuntu' Owner:** Contact the system administrator or the designated owner of the 'ubuntu' account on 'wazuhserver' to determine if this login was legitimate and expected at this time.
  * **Identify Source IP '10.0.1.197':** Determine the device associated with this internal IP address. Is it a known workstation, jump box, or server? Who typically uses this device, and is it authorized to connect to 'wazuhserver' via SSH?
  * **Confirm Purpose:** If legitimate, understand the reason for accessing 'wazuhserver' during non-business hours.

2. **Log Analysis:**
  * **Review 'sshd' Logs on 'wazuhserver':** Look for any preceding failed login attempts, unusual command executions, or other suspicious activity by the 'ubuntu' user immediately before or after this successful login.
  * **Review Logs on '10.0.1.197':** If possible, examine logs from the source machine ('10.0.1.197') for any signs of compromise or unusual activity that might explain the SSH connection.

3. **System Health Check (If Suspicious):**
  * **Check 'wazuhserver':** If the login is deemed unauthorized, or if there's any suspicion, perform a quick security audit on 'wazuhserver'. Check for new user accounts, modified system files, unusual processes, active network connections, or unauthorized cron jobs.
  * **Analyze SSH Keys:** Investigate if the public key used for authentication is still valid, authorized, and not compromised.

4. **Containment & Eradication (If Malicious):**
  * **Disable Account:** Immediately disable the 'ubuntu' user account or revoke its SSH access if the login is confirmed malicious or unauthorized.
  * **Block IP:** Temporarily block '10.0.1.197' from accessing 'wazuhserver' via SSH.
  * **Rotate Keys:** If a public key was compromised, revoke the compromised key and generate new, strong keys, ensuring proper key management.
  * **Isolate Source:** If '10.0.1.197' is compromised, isolate it from the network.

5. **Preventive Measures:**
  * **MFA:** Implement multi-factor authentication (MFA) for SSH access to critical servers like 'wazuhserver'.
  * **IP Whitelisting:** Restrict SSH access to 'wazuhserver' to a limited set of authorized jump servers or IP addresses.
  * **Least Privilege:** Ensure the 'ubuntu' user (or any administrative user) has only the necessary permissions and isn't used for general tasks.
  * **Time-Based Access Control:** Consider implementing stricter time-based access controls for critical systems, beyond just alerting.
  * **Review Business Hours Definition:** Ensure the "non-business hours" definition in Wazuh accurately reflects operational expectations.

6. **False positive probability:**
  **Medium**

  * **Arguments for False Positive:** It is possible for an authorized system administrator, on-call engineer, or developer to genuinely log in outside of regular business hours for maintenance, urgent fixes, or due to being in a different time zone. Public key authentication suggests a pre-configured and usually more secure setup than password-based login.
  * **Arguments against False Positive:** The target ('wazuhserver') is highly critical, the user ('ubuntu') is a common default account, and the early morning time (05:10 AM) is genuinely unusual for most standard operations. The potential for a compromised key or insider threat cannot be ignored.

```

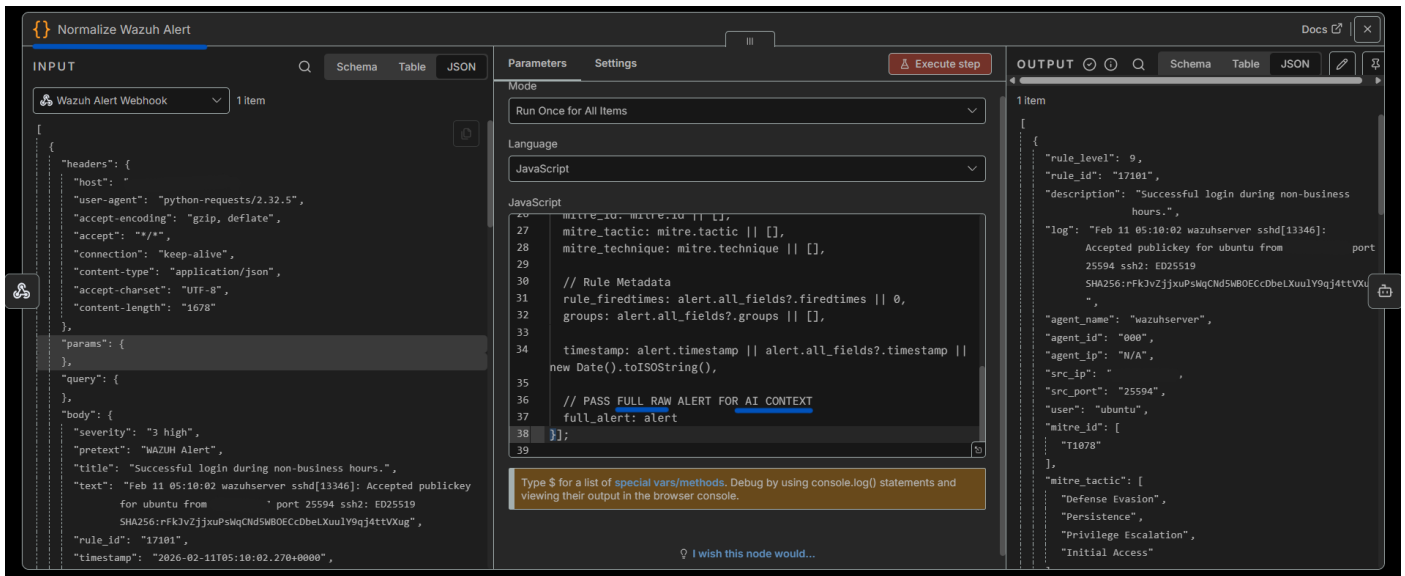
➤ Major Optimization Phase

Step 18: Redesign Alert Context Handling

- Removed unnecessary IF node
- Modified Normalize node
- Passed full raw alert JSON to AI

Result:

AI received full context → Better triage decisions.



The screenshot displays the 'Normalize Wazuh Alert' interface. The 'INPUT' section shows a 'Wazuh Alert Webhook' with 1 item. The 'JSON' tab displays the input alert data, including headers, parameters, and a body with severity, pretext, title, text, rule_id, and timestamp. The 'JavaScript' tab shows the normalization code, which includes a comment: `// PASS FULL RAW ALERT FOR AI CONTEXT`. The 'OUTPUT' section shows the resulting normalized alert, which includes the same input data plus additional fields like 'rule_level', 'rule_id', 'description', 'log', 'agent_name', 'agent_id', 'agent_ip', 'src_port', 'user', 'mitre_id', and 'mitre_tactic'.

Step 19: Advanced Prompt Engineering

Created strict structured prompt including:

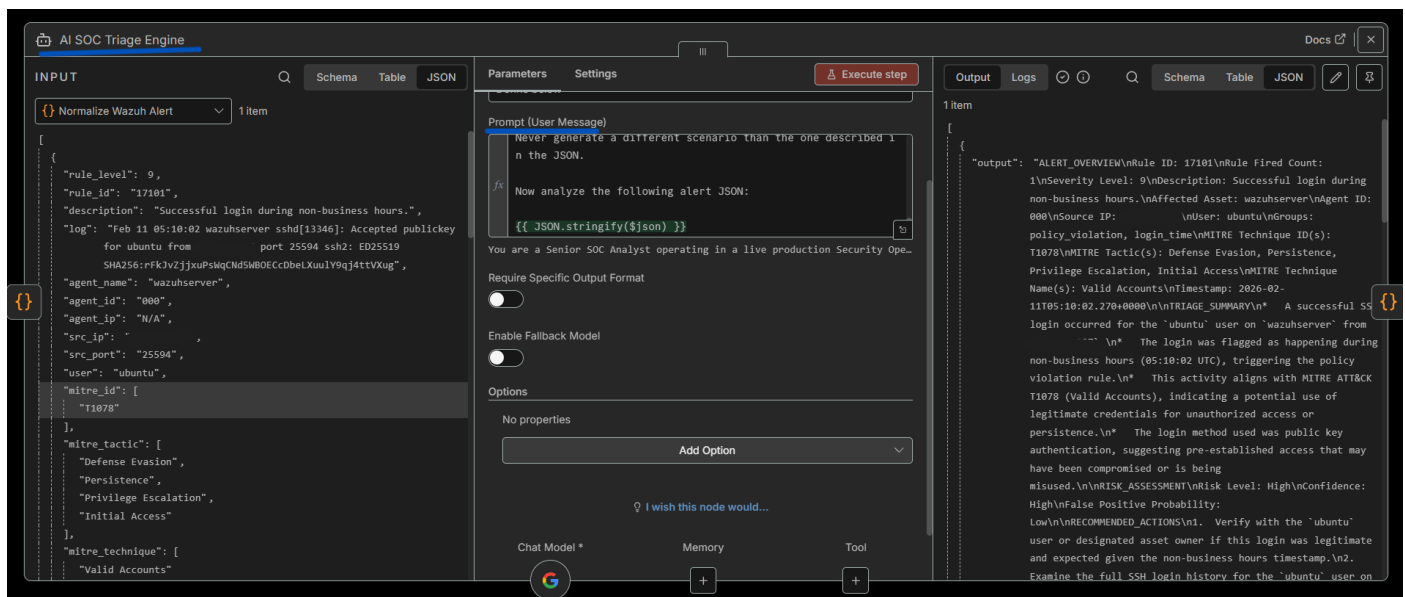
- ALERT_OVERVIEW
- TRIAGE_SUMMARY
- RISK_ASSESSMENT
- RECOMMENDED_ACTIONS
- NEXT_STEP

Added:

- No hallucination rule
- No assumption rule
- Must use only provided JSON
- Professional SOC tone

Result:

Consistent, clean, production-grade output.



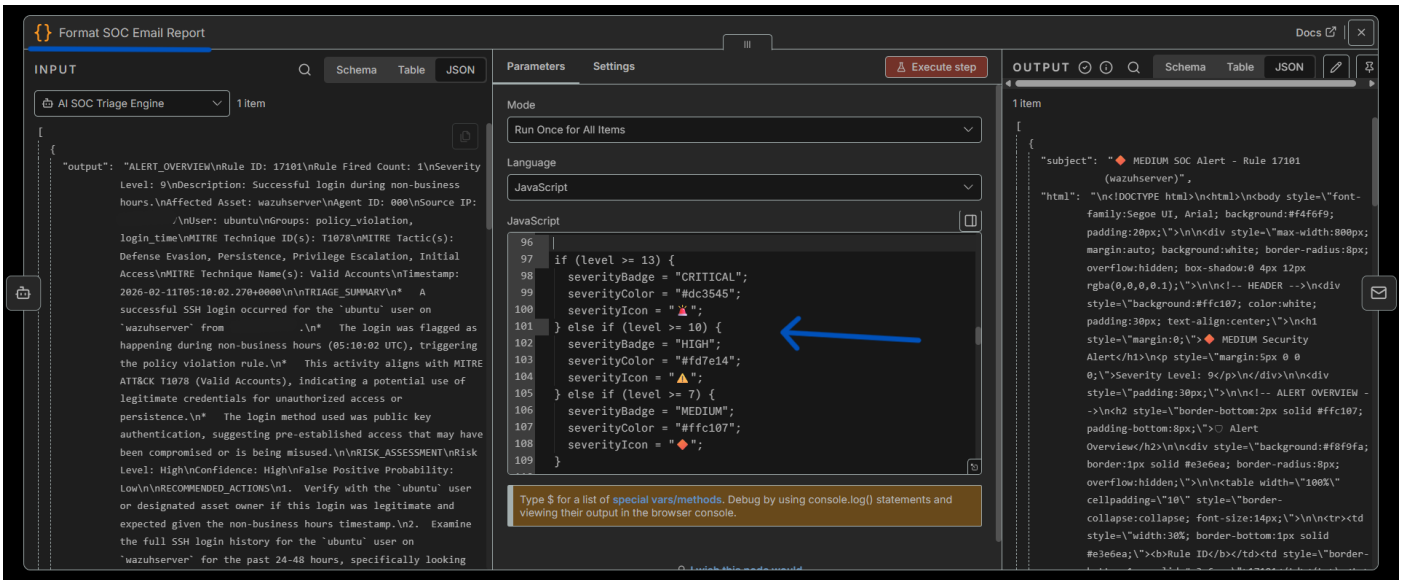
Step 20: Enhanced Email Visual Formatting

Added:

- Severity color coding
- Alert badges (Low / Medium / High / Critical)
- Clean table layout
- Highlighted sections
- Professional SOC design

Purpose:

Improve analyst readability and reduce cognitive load.



Step 21–22: Final Production-Ready Email Output

Final email includes:

❖ Alert Overview

- Rule ID
- Description
- Severity Level
- Rule Fired Count
- Agent Name & ID
- Source IP & Port
- User
- Groups
- MITRE Technique ID
- MITRE Tactic
- MITRE Technique
- Timestamp

❖ **AI Triage Report**

- Executive summary
- Risk Level
- Confidence
- False positive probability
- Recommended actions
- Clear next step

inbox x

@gmail.com

MEDIUM Security Alert Severity Level: 9 Alert Overview Rule ID: 17101 Description: Successful login during non-business hours. Agent: wazuhserver IP Address: [redacted]

1:08 PM (5 hours ago)

☆

@gmail.com

Summary "A successful SSH login occurred for the 'ubuntu' user on 'wazuhserver' from '[redacted]'. " The login was flagged as happening during non-business hours

3:28 PM (3 hours ago)

☆

@gmail.com

to me

3:47 PM (2 hours ago)

☆

MEDIUM Security Alert

Severity Level: 9

Alert Overview

Alert Overview

Rule ID	17101
Description	Successful login during non-business hours.
Severity Level	9
Rule Fired Count	0
Agent Name	wazuhserver
Agent ID	000
Source IP	[redacted]
Source Port	25594
User	ubuntu
Groups	N/A
MITRE Technique ID	T1078
MITRE Tactic	Defense Evasion, Persistence, Privilege Escalation, Initial Access
MITRE Technique	Valid Accounts
Timestamp	2026-02-11T05:10:02.270+0000

Log Details

Feb 11 05:10:02 wazuhserver sshd[13346]: Accepted publickey for ubuntu from [redacted] port 25594
ssh2: ED25519 SHA256:rFkJvZjjxuPswqCnd5WBOECcDbeLXuulY9qj4ttVXug



AI Triage Report

Summary

- * A successful SSH login occurred for the `ubuntu` user on `wazuhserver` from [REDACTED]
- * The login was flagged as happening during non-business hours (05:10:02 UTC), triggering the policy violation rule.
- * This activity aligns with MITRE ATT&CK T1078 (Valid Accounts), indicating a potential use of legitimate credentials for unauthorized access or persistence.
- * The login method used was public key authentication, suggesting pre-established access that may have been compromised or is being misused.

Risk Assessment

Risk Level: High



Recommended Actions

1. Verify with the `ubuntu` user or designated asset owner if this login was legitimate and expected given the non-business hours timestamp.
2. Examine the full SSH login history for the `ubuntu` user on `wazuhserver` for the past 24-48 hours, specifically looking for other unusual logins or activities from [REDACTED] or different source IPs.
3. If the login is confirmed unauthorized, immediately revoke the public key used for access and reset the `ubuntu` user's credentials on `wazuhserver`.
4. If malicious activity is confirmed, isolate `wazuhserver` from the network and escalate to the Incident Response team for further investigation.



Next Step: Validate user activity and proceed with containment if unauthorized.

Project Impact

✓ Reduced Manual Log Analysis

AI summarizes raw logs instantly.

✓ Reduced Alert Fatigue

Only relevant alerts forwarded.

✓ Improved MTTR

Analyst receives:

- What happened
- Why it matters
- What to do next

Real SOAR Implementation

End-to-end automation pipeline built hands-on.

Skills Demonstrated

- SIEM Integration (Wazuh)
- Custom Security Automation Scripting
- SOAR Workflow Design
- AI Prompt Engineering
- JSON Data Structuring
- Email Automation
- Security Alert Triage
- MITRE ATT&CK Mapping Interpretation
- Troubleshooting & Optimization

Learning Outcomes

- Importance of structured prompt engineering
- How AI agents behave without strict constraints
- Real-world SOC automation design principles
- Building scalable AI-assisted security workflows
- End-to-end integration troubleshooting

Conclusion

This project demonstrates a powerful real-world use case of integrating AI into SOC operations.

By combining:

- Wazuh SIEM
- n8n automation
- AI Agent (Gemini)
- Custom scripting

I built a production-style automated triage engine that:

- Reduces analyst workload
- Improves response speed
- Provides structured intelligence
- Demonstrates practical SOAR implementation

This project represents a strong capstone-level AI automation deployment within cybersecurity operations.
